CAPSTONE PROJECT REPORT

SAP SD Order-to-Cash Analytics:

An End-to-End Python Simulation of the O2C Data Pipeline with Enterprise-Grade Analytical Reporting

Name: Pragyan Prayas Jena
Roll Number: 23053538
University: KIIT University, Bhubaneswar

Program: B.Tech Computer Science & Engineering
Course: SAP Data Analytics — Industry Elective
Batch: 2023–2027

300	747	86.1%	14.5d	22.7%
Sales Orders	Line Items	Delivery Rate	Avg Lead Time	On-Time Rate

1. Problem Statement

1.1 Complexity of the Order-to-Cash Lifecycle

In any enterprise operating on SAP, the Order-to-Cash (O2C) process is one of the most operationally dense and financially consequential workflows. It encompasses every touchpoint from the moment a customer places an order to the moment that revenue is recognised in the general ledger. In a manufacturing or retail context, this lifecycle does not follow a single linear sequence; it is a multi-threaded process involving the Sales, Logistics, Warehousing, and Finance departments, each contributing data to a different module of the SAP system.

The process begins in the Sales and Distribution (SD) module when a sales order is created. This order is captured in two linked database tables: VBAK stores the document header — customer identity, sales organisation, document date, and total order value — while VBAP records every individual line item within that order, including material number, quantity, price, and delivery schedule. The data then migrates downstream: the logistics team creates a delivery document (stored in LIKP), which governs the movement of physical goods from warehouse to customer. Finally, once goods are issued, the Finance module generates a billing document (VBRK) which formally records the receivable in the system. Each of these events is time-stamped, and the delta between timestamps is where the critical business intelligence lies.

The core challenge is that this operational intelligence is distributed across four non-trivially joined tables, each in a different functional module with different primary and foreign key conventions. A single order may have five line items, two partial deliveries, and an invoice that only covers a subset of what was shipped. Tracking the complete financial and operational health of any given order requires a precise multi-table join strategy — one that preserves open items rather than silently discarding them, as a naive inner join would.

1.2 Limitations of Standard SAP GUI Reporting

SAP's native reporting layer — primarily the ALV (ABAP List Viewer) Grid — is a transactional tool designed for record-level inspection, not strategic pattern recognition. While ALV grids

are powerful for viewing individual document flows, they suffer from three structural limitations. First, ALV grids present data in tabular form without native charting capability. A business leader asking "which materials are generating the most revenue this quarter?" cannot receive a visual answer from standard SAP transactions like VA05 or VF05 without significant ABAP custom development. Second, cross-module aggregation — computing the elapsed time between VBAK.AUDAT (order creation date) and LIKP.WADAT_IST (actual goods issue date) — requires either a multi-join ABAP SELECT statement or reliance on SAP's Logistics Information System (LIS), both of which demand developer-level access. Third, real-time SLA monitoring across thousands of active orders is computationally expensive inside a production SAP system during business hours.

1.3 The SLA Tracking Challenge

Service Level Agreements in supply chain contexts define the maximum acceptable elapsed time between a customer's order and the delivery of goods. Operational tracking requires comparing VBAK.AUDAT against LIKP.WADAT_IST — a comparison that cannot be made within a single table and requires a join across two modules with different foreign key naming conventions. Furthermore, a meaningful SLA dashboard must handle the case where no delivery has occurred yet: these open-order records are the highest-priority items for a fulfilment manager, yet they are precisely the records that a naive inner join would discard. Simulating and exposing this logic is a central design requirement of this project.

2. Solution Architecture & Key Features

The proposed solution is a self-contained Python analytics pipeline that performs three sequential operations: it simulates realistic SAP SD table data, processes that data through a join strategy that faithfully replicates SAP's relational model, and renders the processed data into three executive-grade visualisations. The pipeline is architected in four distinct stages — Data Generation, Data Processing, KPI Calculation, and Dashboard Rendering — each of which maps directly to a concept that would exist in a real SAP analytics implementation.

2.1 Automated KPI Calculation

Beyond visualisation, the pipeline computes five Key Performance Indicators that a Finance or Supply Chain Controller would monitor daily:

- **Total Order Value (VBAP.NETWR aggregate):** Gross revenue committed across all sales orders, regardless of fulfilment status.
- **Billed Revenue (VBRK.NETWR aggregate):** Revenue formally invoiced and eligible for collection — the basis of Accounts Receivable.
- **Unbilled / Open AR Exposure:** The delta between ordered and billed revenue. At ₹10.25 Cr in the simulation, this is cash trapped in process — goods delivered but not yet invoiced.
- **Average Order-to-Delivery Lead Time:** Computed as (LIKP.WADAT_IST − VBAK.AUDAT) in calendar days. At 14.5 days average against a 7-day SLA, the simulation exposes a systemic fulfilment delay.
- **On-Time Delivery Rate:** Percentage of delivered orders where LEAD_TIME_DAYS ≤ 7. The simulated rate of 22.7% provides an immediately actionable signal for the Logistics team.

3. Data Architecture & Technical Stack

3.1 Technology Stack

Technology	Version / Library	Role in Pipeline
Python 3.11	Core Language	Pipeline orchestration, logic, scripting
pandas 2.x	Data Processing	DataFrame operations, multi-table merges, time arithmetic
NumPy	Numerical	Seeded random data generation ensuring reproducibility
Matplotlib	Visualisation	Primary charting: barh, histogram, dual-axis line chart
GridSpec	Layout Engine	Pixel-precise multi-panel dashboard composition
FuncFormatter	Axis Formatting	Currency-aware axis labels with ₹ M/Cr suffixes

3.2 SAP Table Mapping

The four SAP SD tables simulated in this project, their structural role in the O2C data model, and the fields used as join keys in the analytics pipeline:

Table	Business Entity	Primary Key	Key Fields Used
VBAK	Sales Order Header — one row per complete sales document; the top-level anchor of every O2C transaction.	VBELN	AUDAT (order date), KUNNR (customer), VKORG (sales org), NETWR (net value), WAERK (currency)
VBAP	Sales Order Item — one row per line item within an order; child table to VBAK in a 1:N relationship.	VBELN + POSNR	MATNR (material), MATKL (material group), KWMENG (quantity), NETPR (unit price), NETWR (item revenue)
LIKP	Delivery Header — created by Logistics when goods are allocated for shipment; references parent sales order via VGBEL.	VBELN (delivery)	VGBEL (ref. to VBAK.VBELN), LFDAT (planned delivery), WADAT_IST (actual goods issue), LFART (type)
VBRK	Billing Header — the financial invoice document; links back to the sales order via the ZUONR assignment field.	VBELN (billing)	ZUONR (ref. to VBAK.VBELN), FKDAT (billing date), NETWR (invoiced amount), FKART (billing type)

3.3 Join Strategy — Simulating SAP's Open Items Logic

The most architecturally critical decision in this pipeline is the choice between inner joins and left joins at each merge step. An inner join would silently discard any sales order without a corresponding delivery — precisely the open orders that represent unfulfilled customer commitments. SAP's own reporting philosophy, expressed in transactions like VL06O (Delivery Monitor), explicitly surfaces these open items. The left join replicates this philosophy: every sales order is preserved regardless of whether a matching delivery or

billing record exists. Where no match is found, the delivery and billing fields are populated with NaN, semantically encoding "not yet fulfilled."

The join sequence proceeds in three steps: (1) VBAK inner-joined to VBAP on VBELN — a sales order without line items is structurally invalid; (2) the result left-joined to LIKP on VBAK.VBELN = LIKP.VGBEL, preserving undelivered orders; (3) the result left-joined to VBRK on VBAK.VBELN = VBRK.ZUONR, preserving uninvoiced orders.

```
# Step 1: Header + Items (inner — both must exist for a valid document)
df = vbak.merge(vbap, on='VBELN', how='inner', suffixes=('_HDR', '_ITM'))

# Step 2: Attach delivery (left — preserves orders with no delivery yet)
df = df.merge(likp[['VGBEL','LFDAT','WADAT_IST','LFART']],
              left_on='VBELN', right_on='VGBEL', how='left')

# Step 3: Attach billing (left — preserves delivered-but-not-billed orders)
df = df.merge(vbrk[['ZUONR','FKDAT','NETWR','FKART']],
              left_on='VBELN', right_on='ZUONR', how='left', suffixes=('','_BILL'))
```

3.4 Lead Time Computation — Timestamps to Business Intelligence

One of the most analytically valuable transformations is the conversion of raw datetime columns into scalar business metrics. Both VBAK.AUDAT and LIKP.WADAT_IST are represented as pandas Timestamp objects, making the arithmetic directly vectorised — applied simultaneously across all delivered records without a loop.

```
# Programmatic translation of timestamps into actionable lead time
df['LEAD_TIME_DAYS'] = (df['WADAT_IST'] - df['AUDAT']).dt.days

# Boolean KPIs derived from left-join NaN pattern
df['IS_DELIVERED'] = df['WADAT_IST'].notna()   # False = open order
df['IS_BILLED']    = df['FKDAT'].notna()        # False = delivered, not invoiced
```

The notna() flags transform the NaN-presence pattern — the natural output of a left join with no match — into explicit boolean business indicators. IS_DELIVERED = False identifies open fulfilment items; IS_BILLED = False on a delivered order identifies the AR risk exposure. This is the Python equivalent of SAP's document status management via VBUK/VBUP status tables.

4. Analytics Output & Dashboard

4.1 Dashboard Overview

The analytics dashboard produced by the pipeline consolidates all three visualisations onto a single dark-themed canvas styled to reflect the aesthetic conventions of enterprise BI tools such as SAP Analytics Cloud and Power BI. The dark background reduces eye strain during extended analysis sessions and creates high contrast with the colour-coded data elements. The layout is composed using matplotlib's GridSpec engine, which allows precise control over subplot sizing and spacing.

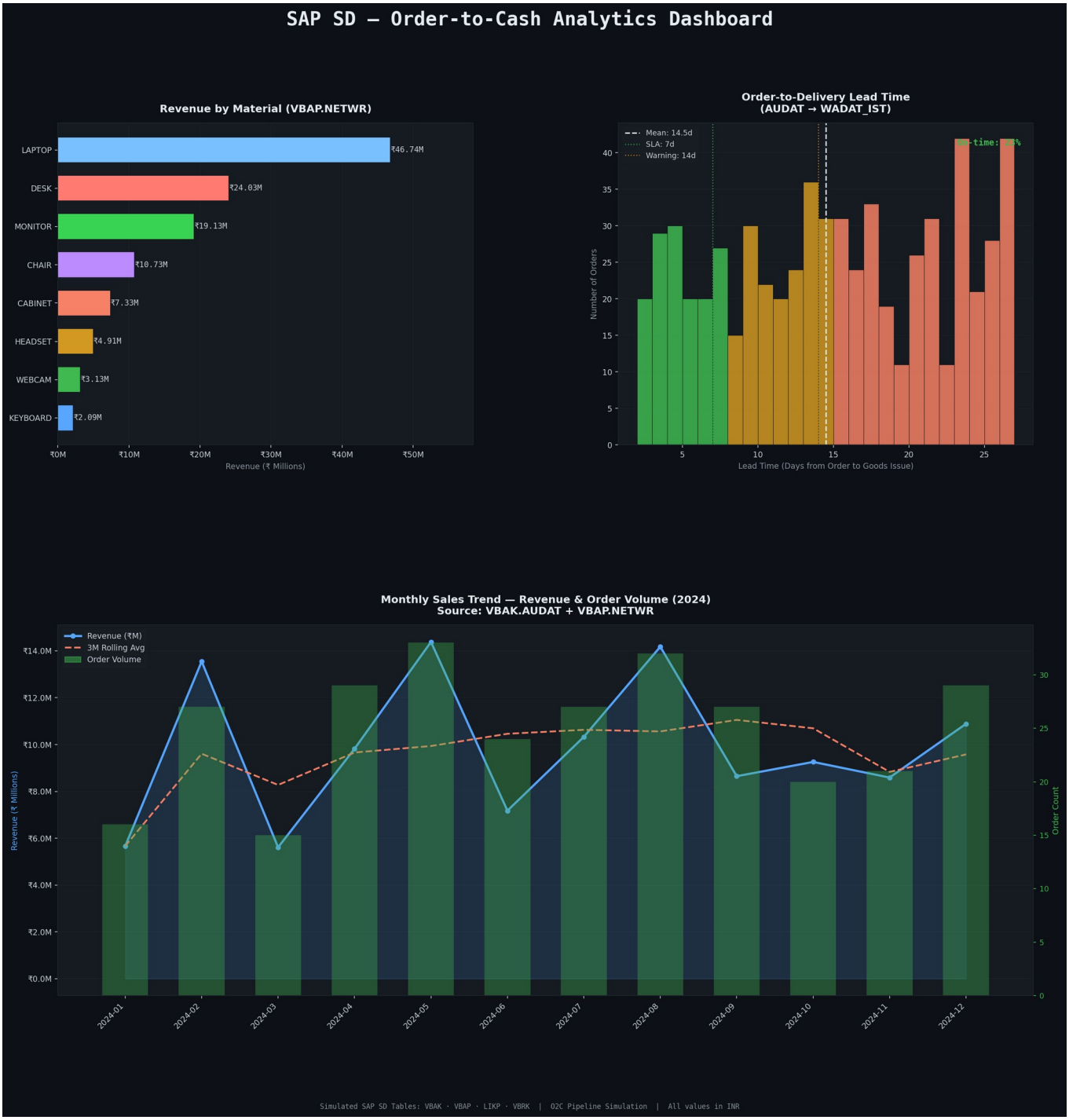

Figure 1: SAP SD Order-to-Cash Analytics Dashboard — Python Simulation Output

4.2 Chart-by-Chart Business Analysis

Chart 1 — Revenue by Material (VBAP.NETWR Aggregation).

This horizontal bar chart answers the most fundamental question in any sales analytics workflow: which products are generating revenue? The data is derived by grouping the line-item table (VBAP) on MATNR and summing NETWR. In a real SAP context, this is the primary output of the SD Information System transaction MC.9 or a custom ALV report on VBAP. The simulation shows LAPTOP as the highest-revenue material at₹46.74M, followed by DESK (₹24.03M) and MONITOR (₹19.13M) — consistent with a B2B technology and office equipment supplier. The colour-coded bars allow a product manager to instantly identify portfolio concentration risk. If LAPTOP represents over 40% of revenue and simultaneously has the longest lead time, that is a critical intersecting risk demanding immediate action.

Chart 2 — Order-to-Delivery Lead Time Distribution (AUDAT → WADAT_IST).

The histogram visualises the distribution of lead times across all 255 delivered orders with a deliberate three-zone SLA colour system: green (≤7 days, on-time), amber (8–14 days, at-risk), and red (>14 days, SLA breach). A vertical dashed line marks the mean at 14.5 days — already in the amber zone — and reference lines at 7 and 14 days make the SLA boundaries visually unambiguous. The on-time delivery rate of 22.7% printed in the chart provides an immediate executive summary. The right-skewed distribution, with a tail extending beyond 20 days, indicates a subset of orders experiencing systemic delay — an insight that would drive the next level of drill-down analysis by material, customer, or logistics partner.

Chart 3 — Monthly Sales Trend with Dual Axis (Revenue & Order Volume).

The dual-axis time series chart is the most analytically nuanced of the three. The primary y-axis tracks monthly revenue (₹M), while the secondary y-axis tracks the count of unique sales orders placed per month. A 3-month rolling average smooths out month-to-month noise to surface the underlying trend direction. The dual-axis design is analytically deliberate: if revenue grows while order volume remains flat, Average Selling Price is increasing — either a product mix shift toward higher-value materials or successful price increases. Conversely, if volume grows but revenue is flat, ASP is declining — a warning sign of discounting pressure. This chart is the Python equivalent of SAP's Sales Information System (transaction MC+2).

5. Unique Design Contributions

5.1 Relational Simulation Without a Live SAP Instance

The simulation of a complex 1-to-many relational environment — where one VBAK record links to multiple VBAP records, which cascade through LIKP and VBRK — is non-trivial to construct without either a live database server or a pre-existing dataset. This project achieves it programmatically using NumPy's seeded random number generation (np.random.seed(42)) to construct per-order item counts, select materials with realistic price distributions, and apply stochastic fulfilment rates (85% delivery, 80% billing of delivered orders) that mirror real-world supply chain performance. The seeded state ensures complete reproducibility — any reviewer running the script produces the identical dataset, join results, and dashboard.

5.2 Faithful Replication of SAP's Foreign Key Conventions

In real SAP SD tables, foreign key references do not follow a universal naming pattern. LIKP references its parent sales order via VGBEL (not VBELN), and VBRK references it via ZUONR (the "assignment number" field used for clearing in FI-AR). These inconsistencies reflect the modular, independently-developed history of SAP's functional areas. The pipeline uses left_on / right_on parameters in the merge operations to explicitly handle these naming asymmetries — the same workaround an ABAP developer would apply when writing a JOIN between tables in different SAP modules. This makes the pipeline architecturally authentic, not merely illustrative.

6. Future Improvements & Enterprise Roadmap

6.1 Live SAP Data Extraction via PyRFC and OData

The natural evolution of this simulation is connection to a live SAP system. SAP exposes its ABAP function modules via RFC (Remote Function Call), which Python can invoke using the

PyRFC library — SAP's officially supported Python connector to the NetWeaver RFC SDK. Using this approach, the data generation phase would be replaced by RFC calls to standard function modules such as RFC_READ_TABLE or custom BAPIs encapsulating the join logic in ABAP. Alternatively, SAP S/4HANA exposes O2C data through pre-built OData services (e.g., API_SALES_ORDER_SRV) consumable via Python's requests library with OAuth2 authentication, making the pipeline accessible without ABAP developer credentials.

6.2 Predictive SLA Breach Forecasting via Machine Learning

The current pipeline is descriptive — it reports what has already happened. The next analytical layer would be predictive: using historical lead time data as training data for a model that forecasts, at order creation time, the probability that a given order will breach the 7-day SLA. Features available at order creation include material category, sales organisation, order quantity, day of week, and customer segment. A gradient-boosted classifier (XGBoost or LightGBM) trained on historical LEAD_TIME_DAYS outcomes would produce a breach probability score for each new order, enabling the Logistics team to proactively allocate expedited fulfilment resources before delays occur.

6.3 SAP HANA Direct Connection for Real-Time Analytics

For organisations running SAP S/4HANA, the Python SAP HANA client library (hdbcli) allows the pipeline to execute join and aggregation logic as a HANA Calculation View query rather than as pandas operations in memory — pushing compute down to the database tier where it is orders of magnitude faster across billions of rows. Combined with a Streamlit or Dash web application layer, the result is a self-refreshing, browser-accessible O2C analytics dashboard requiring no SAP GUI access, distributable to non-technical business stakeholders.

7. Conclusion

This project demonstrates that the analytical intent of enterprise SAP reporting is fully replicable outside the SAP environment using open-source Python tooling, provided the developer understands the underlying relational data model with sufficient depth. The four simulated tables — VBAK, VBAP, LIKP, and VBRK — encode the business logic of the Order-to-Cash cycle: a cascade of events, each dependent on the previous, each timestamped, each linked by keys whose naming reflects the modular history of SAP's development.

The join strategy adopted — inner for the mandatory VBAK-VBAP relationship, left for all downstream fulfilment events — is a deliberate business decision: it ensures that open items, the most operationally critical records in any sales or logistics workflow, are never silently excluded from the analysis. The lead time computation, the SLA colour zones, the dual-axis trend chart, and the unbilled revenue KPI are all expressions of a single underlying principle: the purpose of an analytics system is not to report what happened, but to surface the gap between what was committed and what was delivered.

The simulation achieves a 300-order, 747-item dataset with an 86.1% fulfilment rate and a 22.7% on-time delivery rate — numbers that, in a real enterprise context, would trigger an immediate operational review. The fact that this insight emerges from a Python script running on commodity hardware, without SAP GUI access, without ABAP developer involvement, and without a BI licence, is the project's central contribution: it makes enterprise-grade O2C analytics accessible to any data analyst who understands the relational structure of SAP SD.

Pragyan Prayas Jena · Roll No. 23053538 · B.Tech CSE · KIIT University